

Laboratoire Mobile

Prérequis

- Installer [node.js](#)
- [Android studio](#) simulateur ou un téléphone avec [Expo Go](#).
NB : Attention avec votre téléphone, à l'école, il est nécessaire de partager votre connexion pour le connecter à Expo.
- [Git](#) et un compte [GitHub](#)

Création du dossier GitHub

1. Se connecter à votre compte et accéder à Your repositories
2. Ajouter un nouveau projet 
 - Définissez un nom pour l'url
 - Ajoutez une description si besoin
 - Définissez si votre projet est accessible en public ou en privé
 - N'initialisez pas les documents par défaut, l'init du projet s'en chargera !

Create a new repository

A repository contains all project files, including the revision history. Already have a project repository elsewhere? [Import a repository.](#)

Required fields are marked with an asterisk (*).

Owner * / Repository name *

Navigation is available.

Great repository names are short and memorable. Need inspiration? How about [friendly-octo-eureka](#) ?

Description (optional)

☒  **Public**
Anyone on the internet can see this repository. You choose who can commit.

☐  **Private**
You choose who can see and commit to this repository.

Initialize this repository with:

☐ **Add a README file**
This is where you can write a long description for your project. [Learn more about READMEs.](#)

Add .gitignore

Choose which files not to track from a list of templates. [Learn more about ignoring files.](#)

Choose a license

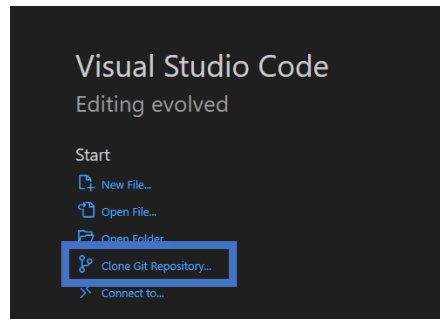
A license tells others what they can and can't do with your code. [Learn more about licenses.](#)

① You are creating a public repository in your personal account.

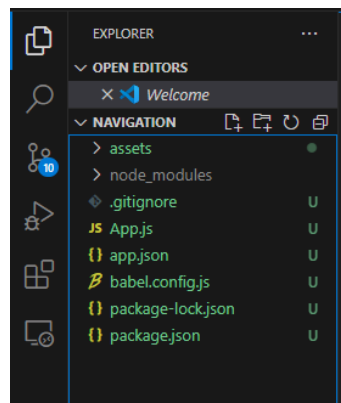
[Create repository](#)

Création de votre projet

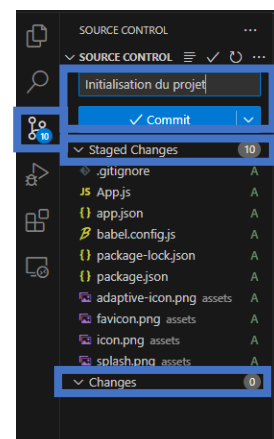
1. Ouvrez votre IDE, Visual Studio ou autre et synchronisez le répertoire créé à l'aide de l'https ou en configurant un accès SSH (cf. cours).



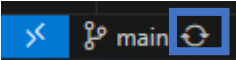
2. Ouvrez un terminal de l'IDE ou accédez en ligne de commande dans le dossier que vous avez synchronisé.
3. Créez votre projet React Native Expo :
 - `npx create-expo-app --template`
 - `Blank`
 - `NomDeVotreProjet`
4. Une fois installé, déplacez l'ensemble de vos fichiers du dossier créé à la racine. Cela vous évitera, à chaque lancement du projet, de devoir changer de dossier pour exécuter les commandes.



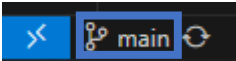
5. Vous remarquez que les documents sont en verts. Cela signifie qu'ils ont été modifiés par rapport à votre branche main distante, sur laquelle vous vous trouvez par défaut. Pour finaliser l'initialisation de votre projet, allez dans le menu Git, ajoutez les fichiers en Stage, à l'aide du + visible au survol du label Change ou `git add --all`, et faites un commit grâce au bouton ou `git commit -m "votreMessage"` avec comme message `Initialisation du projet`.



6. Synchronisez votre dossier vers l'espace Remote à l'aide du bouton en bas à

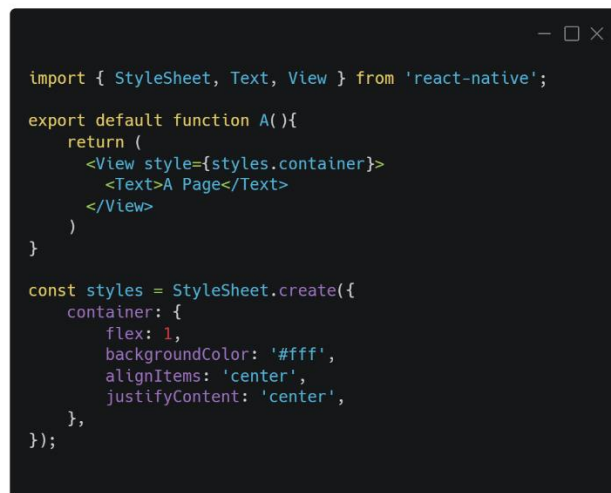
gauche de votre écran  ou `git push`.

7. Félicitation, votre projet a été initialisé ! Il se trouve à présent sur votre répertoire GitHub, vous pouvez directement vérifier sa présence sur le site internet.
8. Pour la suite du projet, nous travaillerons directement sur une nouvelle branche.

Pour créer une branche, cliquez sur le nom de la branche  ou `git checkout -b NomDeVotreBranche`. Une fois votre branche créée, n'oubliez pas de la publier via l'interface ou la ligne de commande `git push --set-upstream origin NomDeVotreBranche`.

Mise en place d'une Bottom-Tabs

1. Pour mettre en place une navigation, vous allez avoir besoin de nouvelles librairies issue de la librairie [React Navigation](#) :
 - `npm install @react-navigation/native`
 - `npx expo install react-native-screens react-native-safe-area-context`
 - Suite à une mise à jour récente, vous rencontrerez peut-être des bugs entre les packages. Dans le dossier GitHub, j'ai inséré des versions fonctionnelles des packages.
 - -> Cela vient d'être patché !
 - `npm install @react-navigation/native-stack`
 - `npm install @react-navigation/bottom-tabs`
2. Pour créer une navigation, il faut créer des pages ! Créez 3 composants dans un dossier screens qui vous serviront de pages de votre application, par exemple :



```
import { StyleSheet, Text, View } from 'react-native';

export default function A(){
  return (
    <View style={styles.container}>
      <Text>A Page</Text>
    </View>
  )
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: '#fff',
    alignItems: 'center',
    justifyContent: 'center',
  },
});
```

3. Il faut maintenant intégrer la navigation. Nous ajouterons deux types de navigation au sein de l'application :
 - Native-Stack : permettant une navigation à l'intérieur des écrans.
 - Bottom-Tabs : permettant d'insérer une barre de menu au bas de votre écran.
4. Commençons par la Bottom-Tabs, pour ce faire, rendez-vous dans la page App.js de votre application et importez les différentes librairies nécessaires ainsi que les pages précédemment créées :
 - `NavigationContainer` : permet de fournir un contexte à votre application pour la navigation, il gère l'état de la navigation, la navigation entre les écrans et l'historique de navigation.
 - `createBottomTabNavigator` : est une fonction permettant de créer le composant Bottom Tabs. Une fois que votre objet est créé, dans la constante Tab dans l'exemple, il pourra prendre différentes props : `initialRouteName` permettant de donner le point de départ, `screenOptions`

permettant ici d'enlever l'entête par défaut. D'autres props sont évidemment disponibles dans la [documentation](#).

```
import { StyleSheet } from 'react-native';

import A from './screens/A';
import B from './screens/B';
import RootNavigator from './screens/Root';

import { NavigationContainer } from '@react-navigation/native';
import { createBottomTabNavigator } from '@react-navigation/bottom-tabs';
const Tab = createBottomTabNavigator();

export default function App() {
  return (
    <NavigationContainer>
      <Tab.Navigator initialRouteName="A" screenOptions={{ headerShown: false }}>
        <Tab.Screen name="A" component={A} />
        <Tab.Screen name="B" component={B} />
        <Tab.Screen name="Root" component={RootNavigator} />
      </Tab.Navigator>
    </NavigationContainer>
  );
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: '#fff',
    alignItems: 'center',
    justifyContent: 'center',
  },
});
```

5. Démarrez votre application `npm run start` et découvrez votre Bottom-Tabs qui vous permet de naviguer à travers les écrans !

Navigation interne

1. Nous avons les différentes parties de notre application, mais il est bien souvent nécessaire de naviguer à l'intérieur même d'un écran. Pour ce faire, nous allons avoir besoin de créer une nouvelle stack de navigation. Il s'agira de créer un nouveau composant permettant de naviguer de A à B et qui sera intégré à notre Bottom-Tabs.
2. Créez votre nouveau composant dans un dossier components et importez les écrans A et B ainsi que les librairies nécessaires :
 1. `createNativeStackNavigator` : fonction permettant de créer une navigation interne à une page. Une fois votre objet créé, vous pourrez également profiter de ses props que vous retrouverez dans la [documentation](#).

```
import A from './screens/A';
import B from './screens/B';

import { createNativeStackNavigator } from '@react-navigation/native-stack';
const Stack = createNativeStackNavigator();

export default function TabNavigator () {
  return (
    <Stack.Navigator
      initialRouteName="Tabs/A"
      screenOptions={{
        headerShown: false,
      }}>
      <Stack.Screen
        initialParams={{
          screen: 'Tabs/A',
        }}
        options={{
          unmountOnBlur: true,
        }}
        name="A"
        component={A}
      />
      <Stack.Screen
        initialParams={{
          screen: 'Tabs/B',
        }}
        options={{
          unmountOnBlur: true,
        }}
        name="B"
        component={B}
      />
    </Stack.Navigator>
  )
}
```

3. Pour naviguer entre les écrans, il faut maintenant créer les liens dans les pages A et B. Vous devrez importer dans ces pages un hooks propre à react-navigation qui donnera l'instruction de naviguer vers le nom de la page que vous lui donnez et qui est référencé dans votre navigation créée précédemment.

```
import { StyleSheet, Text, View, Button } from 'react-native';
import { useNavigation } from '@react-navigation/native';

export default function A(){
  const navigation = useNavigation();

  return (
    <View style={styles.container}>
      <Text>A Page</Text>
      <Button title="To B" onPress={() => navigation.navigate('B')} />
    </View>
  )
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: '#fff',
    alignItems: 'center',
    justifyContent: 'center',
  },
});
```

4. Pour terminer, adapter votre Bottom-Tabs en important le composant nouvellement créé et en remplaçant les tabs A et B par un seul Tab contenant le votre navigation interne.

```
import { StyleSheet } from 'react-native';

import TabNavigator from '../components/TabNavigator';
import RootNavigator from '../screens/Root';

import { NavigationContainer } from '@react-navigation/native';
import { createBottomTabNavigator } from '@react-navigation/bottom-tabs';
const Tab = createBottomTabNavigator();

export default function App() {
  return (
    <NavigationContainer>
      <Tab.Navigator initialRouteName="Tabs" screenOptions={{ headerShown: false }}>
        <Tab.Screen name="Tabs" component={TabNavigator} />
        <Tab.Screen name="Root" component={RootNavigator} />
      </Tab.Navigator>
    </NavigationContainer>
  );
}

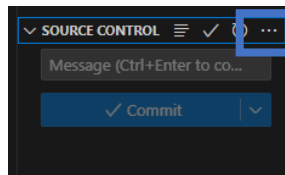
const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: '#fff',
    alignItems: 'center',
    justifyContent: 'center',
  },
});
```

5. Démarrez votre application `npm run start` et découvrez votre Native-Stack qui vous permet de naviguer dans un même écran ! Finalisez cette étape en publiant vos modifications sur le git.

Synchroniser sur la main

Comme présenté lors des cours, une fois que vous avez une version stable, il est temps de vous synchroniser avec la branche main !

1. Commencez par faire un git > Pull, Push > Pull from ... > origin/main de la branche main sur votre branche ou [git pull origin main](#).
 - S'il n'y a pas de conflit, vous avez obtenu les derniers changements.
 - S'il y a des conflits, résolvez-les et une fois terminé, pour être certain, récupérez à nouveau la branche main.



2. Changez de branche et allez sur la main, en cliquant sur le nom de la branche en bas à gauche et en cliquant sur la branche souhaitée ou [git checkout main](#).
3. Faites un pull, mais cette fois, de la branche que vous venez de valider. Encore une fois, depuis l'interface ou [git pull origin NomDeVotreBranche](#). Attention, ne vous trompez pas de branche !
4. Synchroniser les changements avec le GitHub distant avec l'interface ou [git push](#).
5. Retourner sur votre branche de développement par l'interface ou [git checkout NomDeVotreBranche](#).